

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ПРАКТИЧЕСКАЯ РАБОТА 5

По дисциплине Проектирование и реализация баз данных

Тема работы Управление транзакциями

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Войтюк Т.Е.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ЗАДАНИЕ 1	4
2 ЗАДАНИЕ 2	9
3 ЗАДАНИЕ 3	16
4 ЗАДАНИЕ 4	18
5 ЗАДАНИЕ 5	21
ЗАКЛЮЧЕНИЕ	24

ВВЕДЕНИЕ

В отчёте представлено решение практической работы.

Цель – Изучить создание и использование транзакций, блокировки и уровни изоляции в PostgreSQL.

Задачи

- Изучить создание транзакции COMMIT и ROLLBACK.
- Научиться выполнять поиск и обнаружение блокировок.
- Изучить различные уровни изоляции: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE.

Перед выполнением заданий в настройках был отключён auto commit. PgAdmin4 не умеет выводить несколько результатов SELECT, поэтому запуски скриптов были разбиты по инструкциям выборки.

1 ЗАДАНИЕ 1

Транзакция с фиксацией

Первым делом, было проверено выполнение транзакции с фиксацией (рисунки 1 - 3). Как и ожидалось, после изменения в транзакции данные изменились, после фиксации – остались изменёнными.

Query Query History

1

-- Посмотрим значение до изменения

2

SELECT 'Before' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

3

FROM "EmployeesDepartments"."EMPLOYEES"

4

WHERE "EMPLOYEE_ID" = 101;

Data Output Messages Notifications

SQL

Showing rows: 1 to 1

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	Before	101	Neena	Kochhar	21670.00

Рисунок 1 — Просмотр данных до транзакции

Query

Query History

1

-- Начинаем транзакцию

2

BEGIN;

3

-- Повышаем зарплату на 10% сотруднику с ID 101

4

UPDATE "EmployeesDepartments". "EMPLOYEES"

5

SET "SALARY" = "SALARY" * 1.1

6

WHERE "EMPLOYEE_ID" = 101;

7

-- Сморим значение внутри транзакции (до коммита)

8

SELECT 'During' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

9

FROM "EmployeesDepartments"."EMPLOYEES"

10

WHERE "EMPLOYEE_ID" = 101;

Data Output

Messages

Notifications

Рисунок 2 — Изменение и просмотр данных в транзакции

Query

Query History

1

-- Фиксация изменений

2

COMMIT;

3

-- Проверяем итоговое значение

4

SELECT 'After' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

5

FROM "EmployeesDepartments"."EMPLOYEES"

6

WHERE "EMPLOYEE_ID" = 101;

Data Output

Messages

Notifications

Showing rows: 1

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	After	101	Neena	Kochhar	23837.00

Рисунок 3 — COMMIT и просмотр данных после транзакции

Транзакция с откатом

Затем, было проверено выполнение транзакции с откатом (рисунки 4 - 6). Как и ожидалось, после изменения в транзакции данные изменились, а после отказа – вернулись в изначальное состояние.

Query

Query History

1

-- Значения до изменений

2

SELECT 'Before' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

3

FROM "EmployeesDepartments"."EMPLOYEES"

4

WHERE "EMPLOYEE_ID" <= 103

5

ORDER BY 2;

Data Output

Messages

Notifications

Рисунок 4 — Просмотр данных до транзакции

Query

Query History

1

-- Начинаем транзакцию

2

BEGIN;

3

-- Повышаем зарплату всем сотрудникам до ID 103

4

UPDATE "EmployeesDepartments"."EMPLOYEES"

5

SET "SALARY" = "SALARY" * 1.1

6

WHERE "EMPLOYEE_ID" <= 103;

7

-- Значения внутри транзакции, до отката

8

SELECT 'Within transaction' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

9

FROM "EmployeesDepartments"."EMPLOYEES"

10

WHERE "EMPLOYEE_ID" <= 103

11

ORDER BY 2;

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

🗑️

📄

⬇️

📈

SQL

Showing rows: 1 to 4

✎

 Page 1

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	Within transaction	100	Steven	King	29040.00
2	Within transaction	101	Neena	Kochhar	26220.70
3	Within transaction	102	Lex	De Haan	19800.00
4	Within transaction	103	Alexander	Hunold	9900.00

Рисунок 5 — Изменение и просмотр данных в транзакции

Query

Query History

1

-- Откат транзакции

2

ROLLBACK;

3

-- Проверяем, что изменения не сохранены

4

SELECT

5

'After' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

6

FROM "EmployeesDepartments"."EMPLOYEES"

7

WHERE "EMPLOYEE_ID" <= 103

8

ORDER BY 2;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗑️

📄

⬇️

📈

SQL

Showing rows: 1 to 4 ✎ Page 1

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	After	100	Steven	King	26400.00
2	After	101	Neena	Kochhar	23837.00
3	After	102	Lex	De Haan	18000.00
4	After	103	Alexander	Hunold	9000.00

Рисунок 6 — ROLLBACK и просмотр данных после транзакции

Транзакция с точками сохранения

Наконец, были испытаны точки сохранения (рисунки 7 - 11). После отката к «sp2» данные вернулись к состоянию на момент объявления точки сохранения.

Query

Query History

1

BEGIN;

2

SELECT 'Trans started' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

3

FROM "EmployeesDepartments"."EMPLOYEES"

4

WHERE "EMPLOYEE_ID" = 104;

Data Output

Messages

Notifications

Рисунок 7 — Просмотр данных до изменения в транзакции

Query

Query History

1

-- Первая точка сохранения с именем sp1

2

SAVEPOINT sp1;

3

UPDATE "EmployeesDepartments"."EMPLOYEES"

4

SET "SALARY" = 3000

5

WHERE "EMPLOYEE_ID" = 104;

6

7

SELECT 'After SAVEP sp1' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

8

FROM "EmployeesDepartments"."EMPLOYEES"

9

WHERE "EMPLOYEE_ID" = 104;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	After SAVEP sp1	104	Bruce	Ernst	3000.00

Рисунок 8 — SAVEPOINT sp1, изменение и просмотр данных

Query

Query History

1

-- Вторая точка сохранения с именем sp2

2

SAVEPOINT sp2;

3

UPDATE "EmployeesDepartments"."EMPLOYEES"

4

SET "SALARY" = 10000

5

WHERE "EMPLOYEE_ID" = 104;

6

7

SELECT 'After SAVEP sp2' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

8

FROM "EmployeesDepartments"."EMPLOYEES"

9

WHERE "EMPLOYEE_ID" = 104;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1

✎

P

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	After SAVEP sp2	104	Bruce	Ernst	10000.00

Рисунок 9 — SAVEPOINT sp2, изменение и просмотр данных

Query

Query History

1

-- Откат только до второго savepoint

2

ROLLBACK TO SAVEPOINT sp2;

3

4

SELECT 'After ROLLB sp2' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

5

FROM "EmployeesDepartments"."EMPLOYEES"

6

WHERE "EMPLOYEE_ID" = 104;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1

✎

P

	stage	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
	text	[PK] integer	character varying (20)	character varying (25)	numeric (8,2)
1	After ROLLB sp2	104	Bruce	Ernst	3000.00

Рисунок 10 — Возврат к SAVEPOINT sp2, просмотр данных

Query

Query History

1

UPDATE "EmployeesDepartments"."EMPLOYEES"

2

SET "SALARY" = "SALARY"+100

3

WHERE "EMPLOYEE_ID" = 104;

4

5

-- Фиксация полной транзакции

6

COMMIT;

7

8

SELECT 'After trans' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"

9

FROM "EmployeesDepartments"."EMPLOYEES"

10

WHERE "EMPLOYEE_ID" = 104;

Data Output

Messages

Notifications

Рисунок 11 — Изменение, фиксация и просмотр данных

2 ЗАДАНИЕ 2

Строго по заданию были выполнены все необходимые действия, в процессе работы были изучены состояния блокировок. Результат работы представлен на рисунках 12 - 23.

The screenshot displays a database management interface. The top section, titled 'Query', shows a sequence of SQL commands: dropping a table if it exists, creating a new table with two columns (id and price), inserting three rows of data, and finally selecting all data from the table. The bottom section, titled 'Data Output', shows the resulting table with three rows of data. The table has two columns: 'id' (integer, primary key) and 'price' (numeric, 10,2). The data rows are (1, 10.00), (2, 20.00), and (3, 30.00).

```
1 DROP TABLE IF EXISTS public.t1;
2
3 CREATE TABLE public.t1 (
4     id int PRIMARY KEY,
5     price numeric(10,2)
6 );
7
8 INSERT INTO public.t1 VALUES
9     (1, 10.00),
10    (2, 20.00),
11    (3, 30.00);
12
13 SELECT * FROM public.t1;
```

	id [PK] integer	price numeric (10,2)
1	1	10.00
2	2	20.00
3	3	30.00

Рисунок 12 — Создание и заполнение таблицы

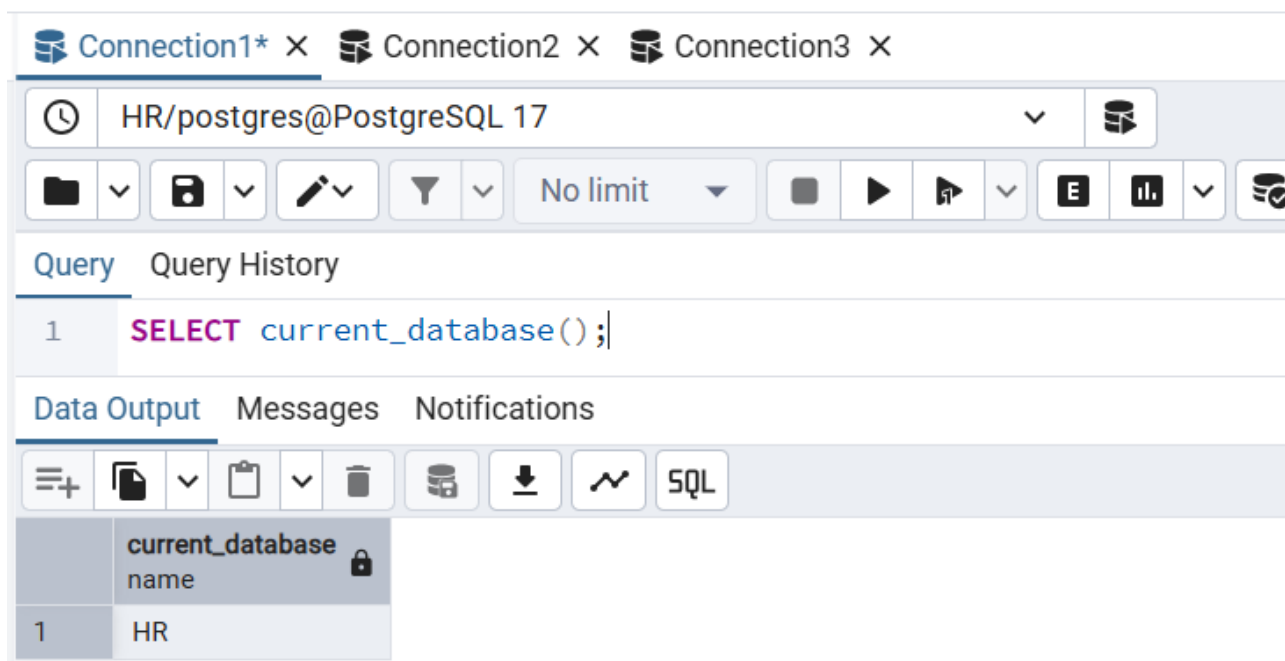


Рисунок 13 — Создание трёх подключений

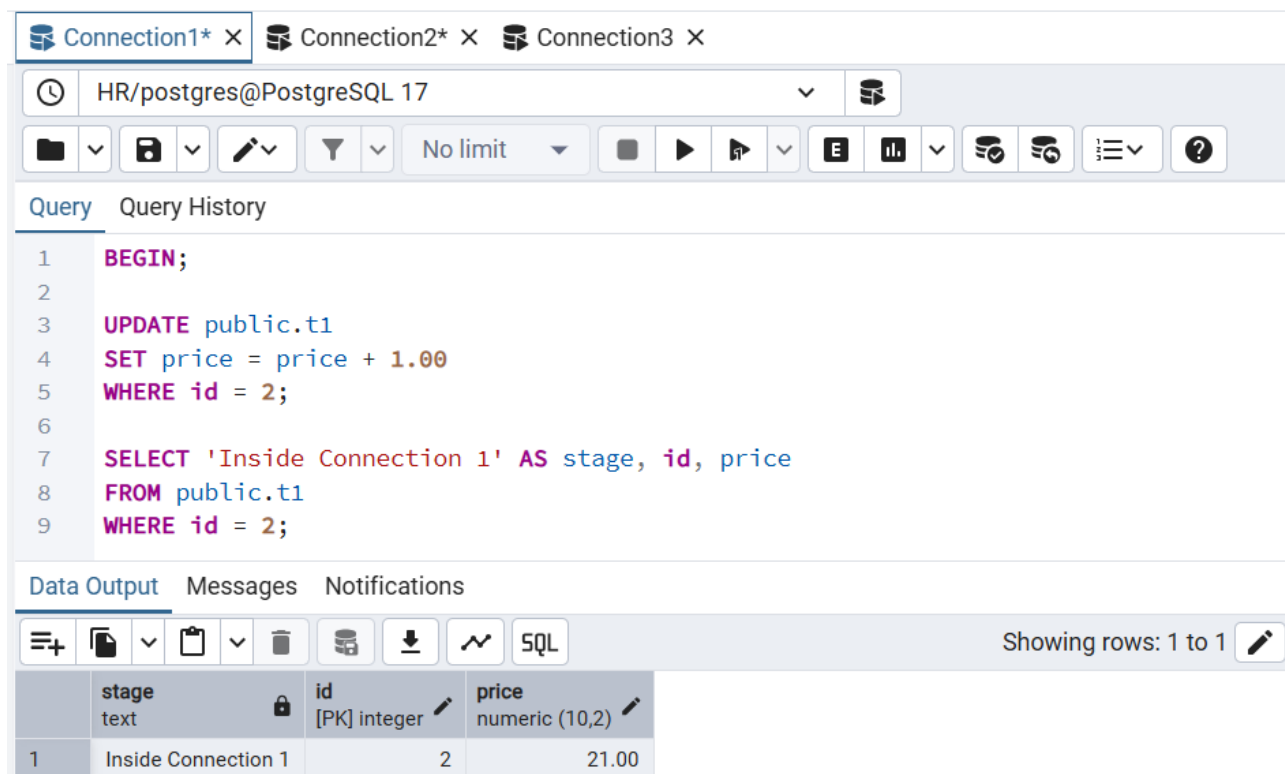


Рисунок 14 — Создание транзакции в первом подключении

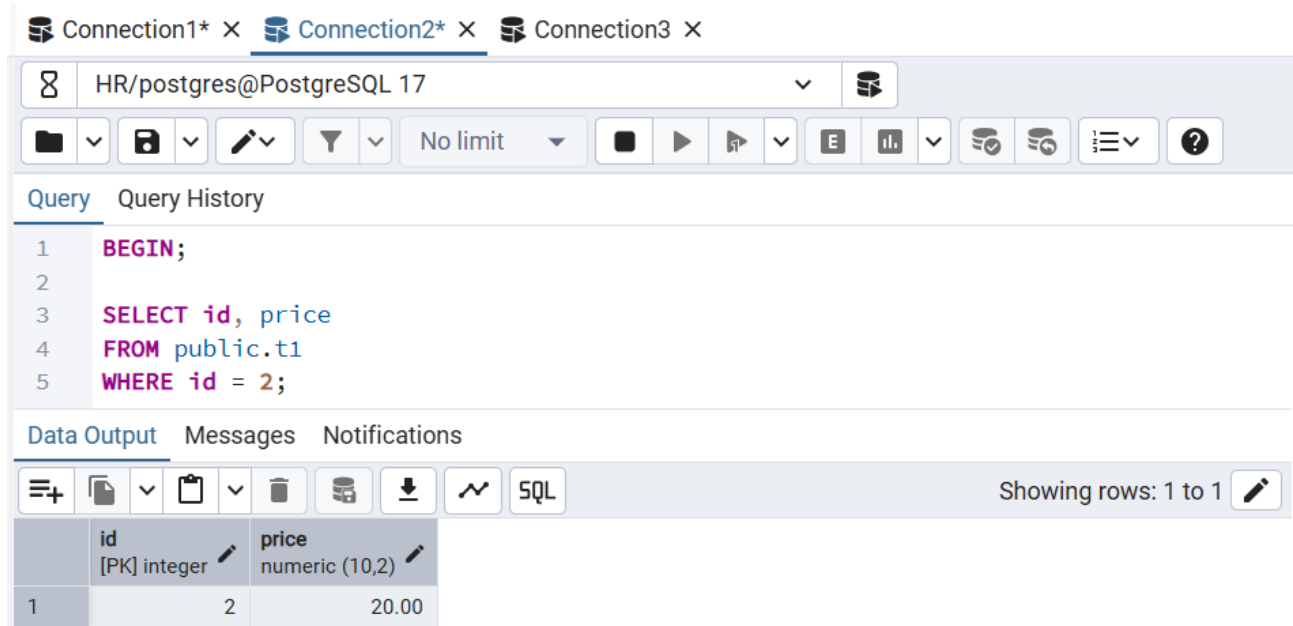


Рисунок 15 — Первый SELECT во втором подключении

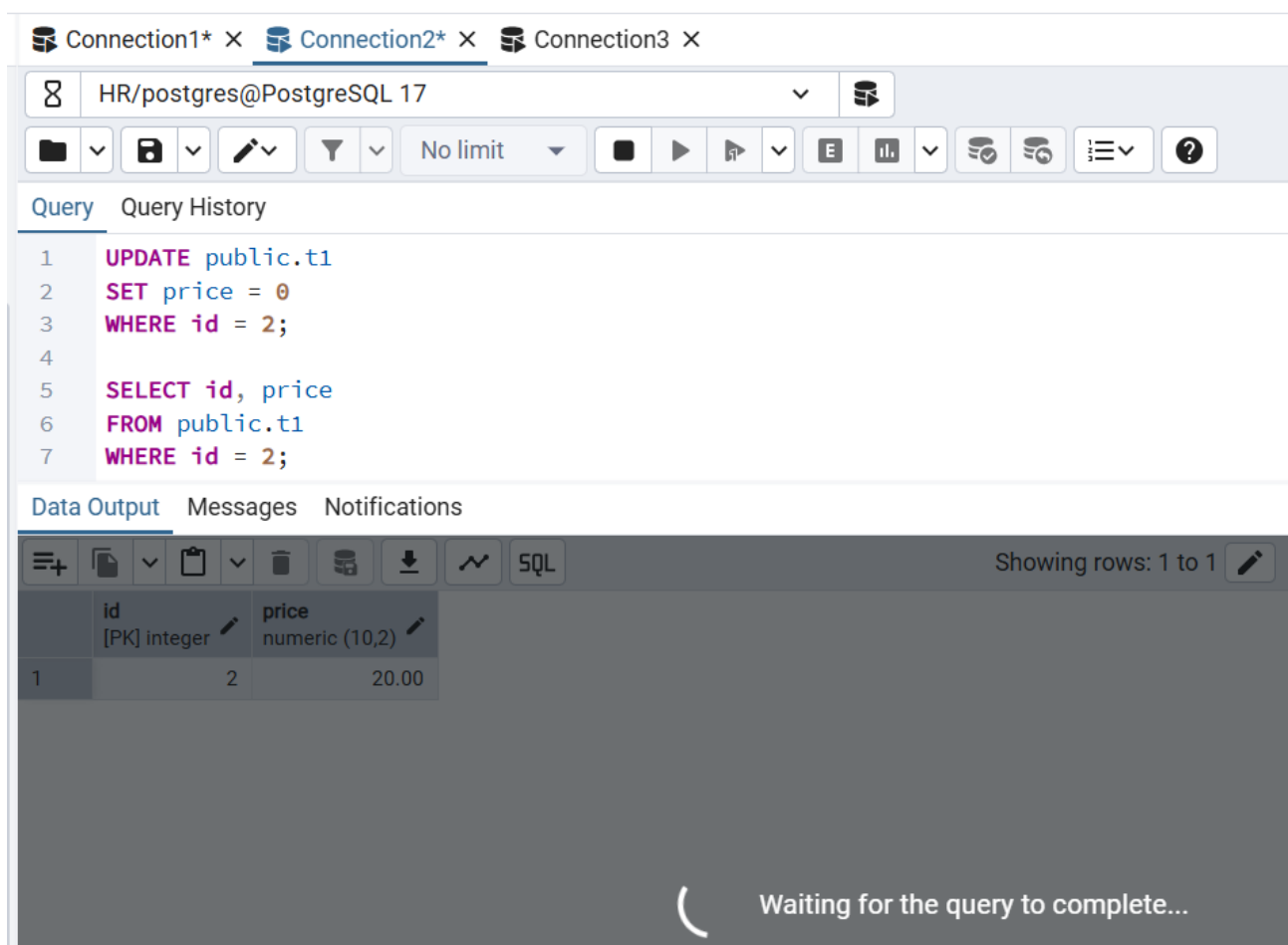


Рисунок 16 — Попытка обновить данные

Connection1* × Connection2* × Connection3* ×

HR/postgres@PostgreSQL 17

Query Query History

```

1 SELECT pid, username, application_name, state, wait_event_type, wait_event, query
2 FROM pg_stat_activity
3 WHERE datname = current_database()
4 ORDER BY pid;

```

Data Output Messages Notifications

Showing rows: 1 to 5

	pid integer	username name	application_name text	state text	wait_event_type text	wait_event text	query text
1	3616	postgres	pgAdmin 4 - CONN:7897650	active	Lock	transactionid	UPDATE public.t1
2	18960	postgres	pgAdmin 4 - CONN:4314474	idle	Client	ClientRead	SELECT
3	28404	postgres	pgAdmin 4 - CONN:8522051	idle in transaction	Client	ClientRead	SELECT oid, pg_c
4	33752	postgres	pgAdmin 4 - DB:HR	idle	Client	ClientRead	SELECT rel.oid, re
5	35740	postgres	pgAdmin 4 - CONN:3362437	active	[null]	[null]	SELECT

Рисунок 17 — Просмотр состояния всех подключений

Connection1* × Connection2* × Connection3* ×

HR/postgres@PostgreSQL 17

Query Query History

```

1 SELECT l.pid, a.application_name, a.state, l.locktype, l.relation::regclass AS locked_relation,
2       l.page, l.tuple, l.virtualxid, l.transactionid, l.mode, l.granted
3 FROM pg_locks l
4 LEFT JOIN pg_stat_activity a ON a.pid = l.pid
5 WHERE a.datname = current_database()
6 ORDER BY l.pid, l.locktype;

```

Data Output Messages Notifications

Showing rows: 1 to 55 Page No: 1 of 1

	pid integer	application_name text	state text	locktype text	locked_relation regclass	page integer	tuple smallint	virtualxid text	transactionid xid
1	3616	pgAdmin 4 - CONN:7897650	active	relation	t1	[null]	[null]	[null]	[
2	3616	pgAdmin 4 - CONN:7897650	active	relation	t1_pkey	[null]	[null]	[null]	[
3	3616	pgAdmin 4 - CONN:7897650	active	relation	t1_pkey	[null]	[null]	[null]	[
4	3616	pgAdmin 4 - CONN:7897650	active	relation	t1	[null]	[null]	[null]	[
5	3616	pgAdmin 4 - CONN:7897650	active	transactionid	[null]	[null]	[null]	[null]	[
6	3616	pgAdmin 4 - CONN:7897650	active	transactionid	[null]	[null]	[null]	[null]	[
7	3616	pgAdmin 4 - CONN:7897650	active	tuple	t1	0	5	[null]	[
8	3616	pgAdmin 4 - CONN:7897650	active	virtualxid	[null]	[null]	[null]	69/9	[
9	28404	pgAdmin 4 - CONN:8522051	idle in transaction	relation	pg_attribute_relid_attnum_index	[null]	[null]	[null]	[
10	28404	pgAdmin 4 - CONN:8522051	idle in transaction	relation	pg_attribute_relid_attnam_index	[null]	[null]	[null]	[
11	28404	pgAdmin 4 - CONN:8522051	idle in transaction	relation	pg_namespace	[null]	[null]	[null]	[

Рисунок 18 — Просмотр состояния всех блокировок

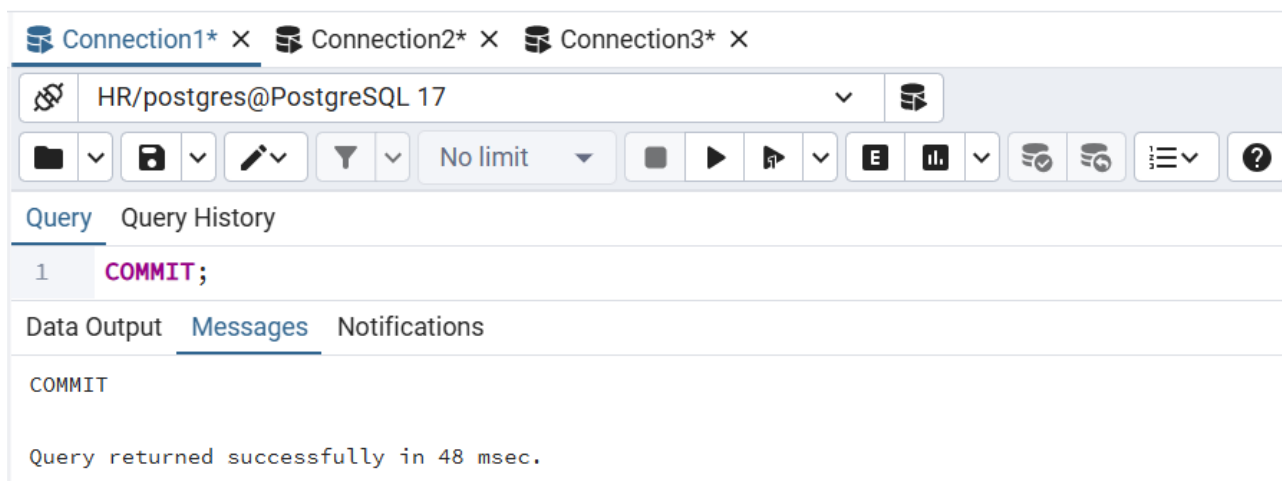


Рисунок 19 — Завершение транзакции в первом подключении

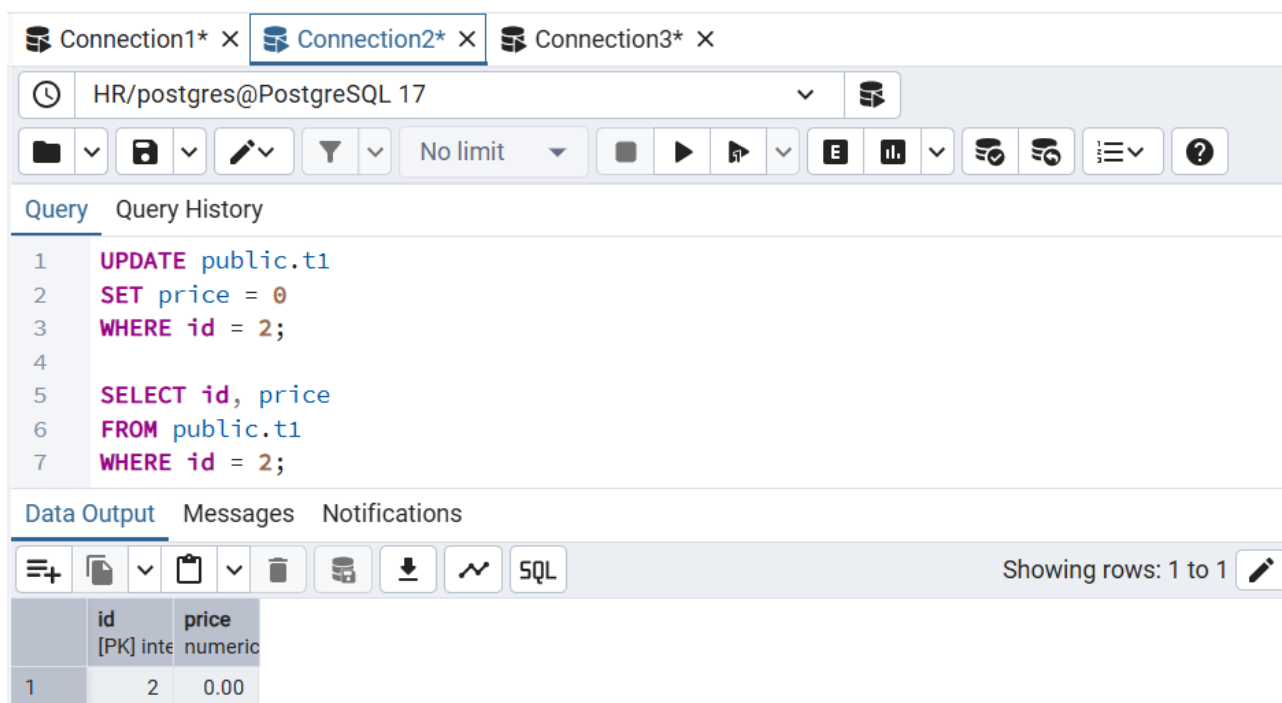


Рисунок 20 — Обновление данных смогло выполниться

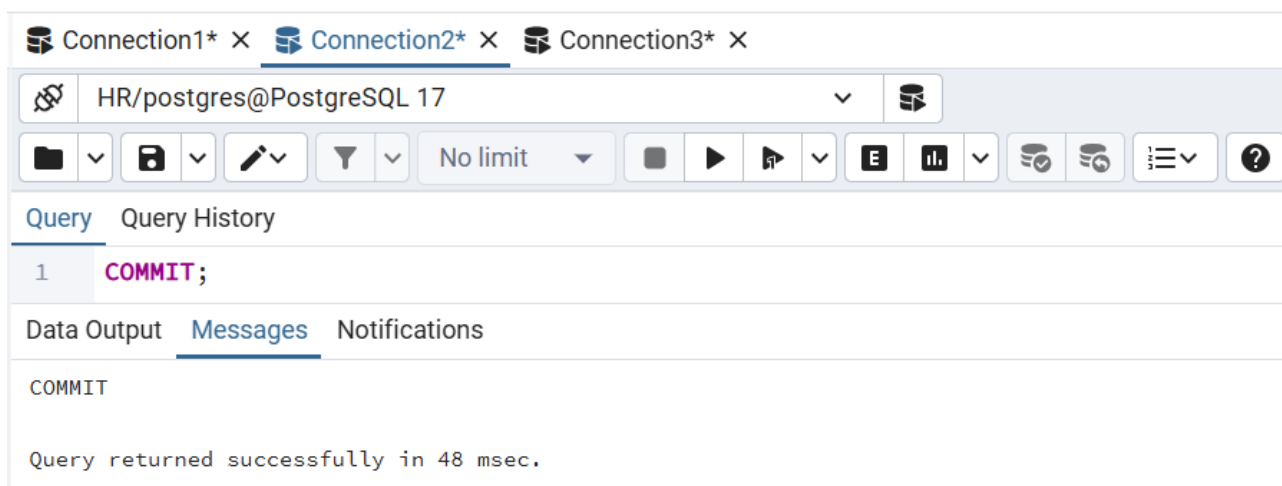


Рисунок 21 — Завершение транзакции во втором подключении

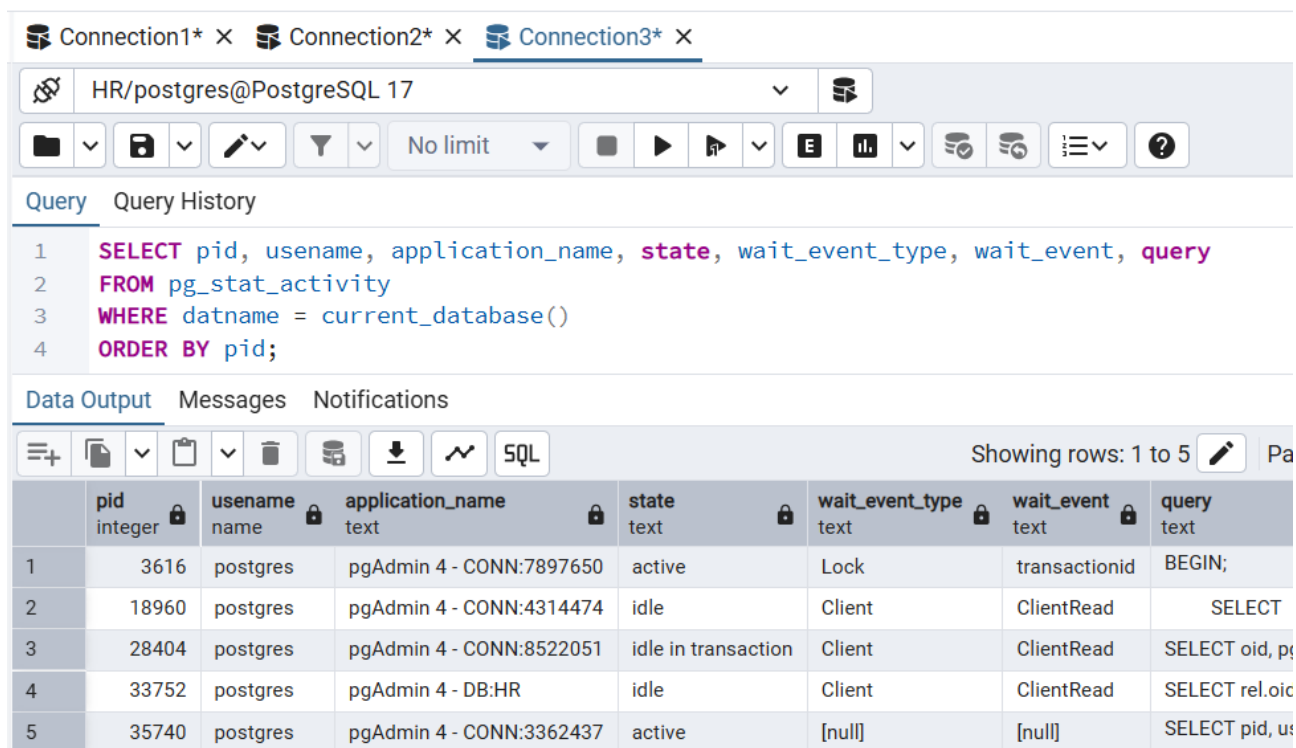


Рисунок 22 — Просмотр состояния всех подключений

3 ЗАДАНИЕ 3

В этом задании был рассмотрен уровень изоляции READ COMMITTED. Результаты выполнения задания представлены на рисунках 24 - 28.

Как и ожидалось, разные операторы SELECT в рамках одной транзакции могут видеть разные данные, если между их выполнениями другая транзакция зафиксировала изменения.

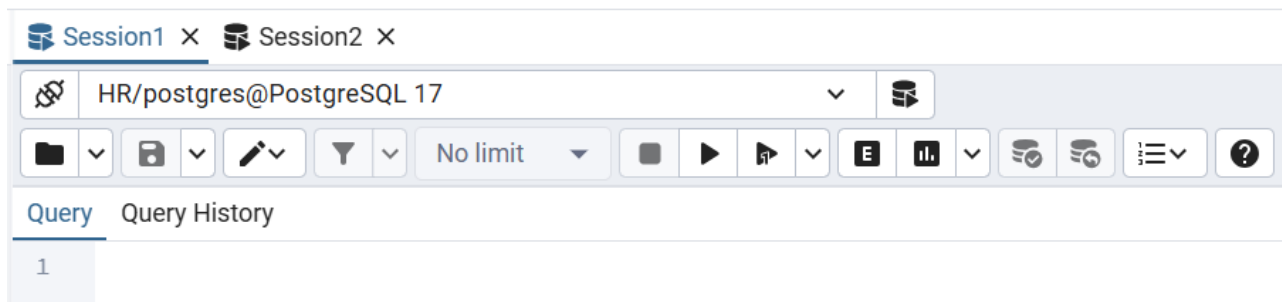


Рисунок 24 — Созданы 2 подключения

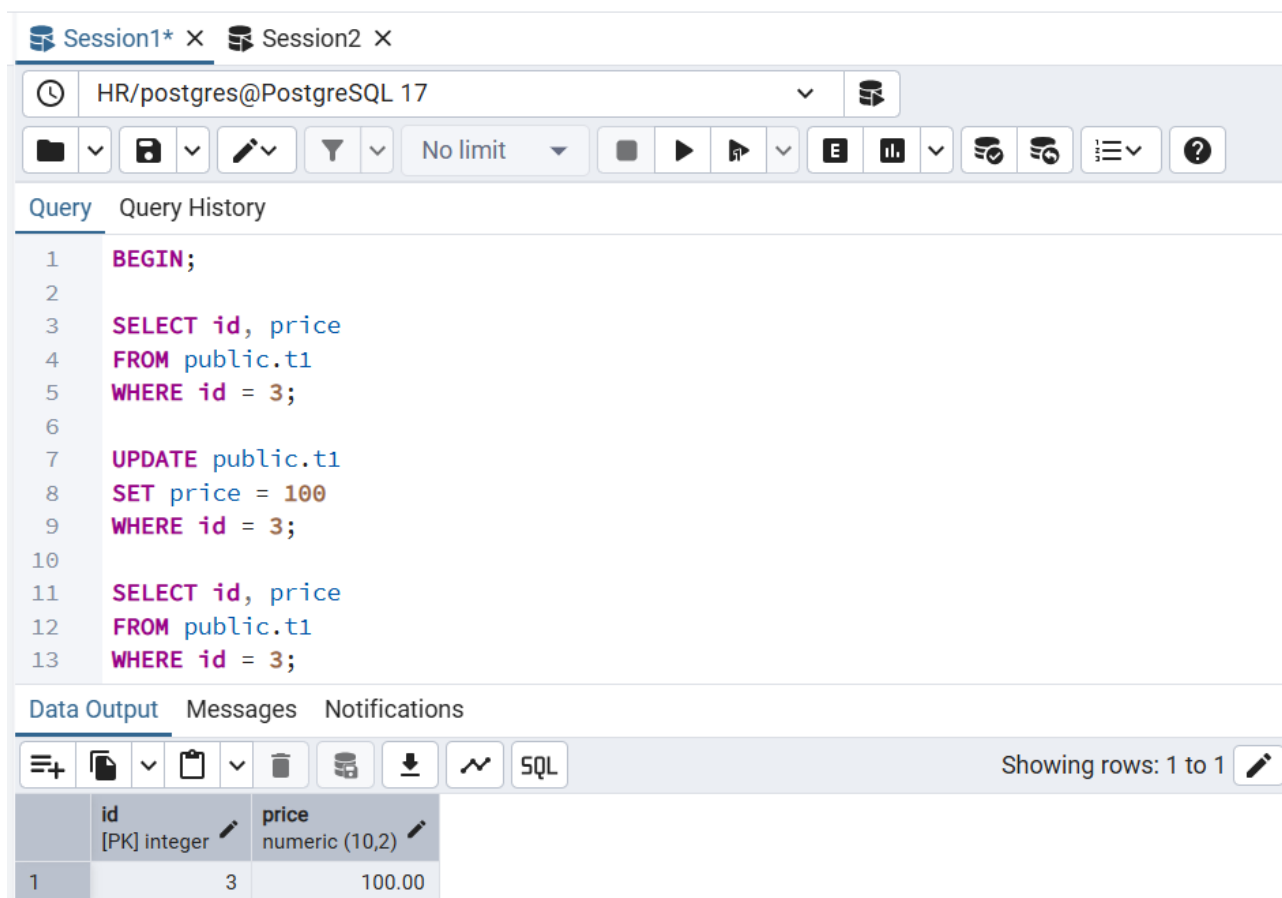


Рисунок 25 — Начало первой транзакции и обновление данных

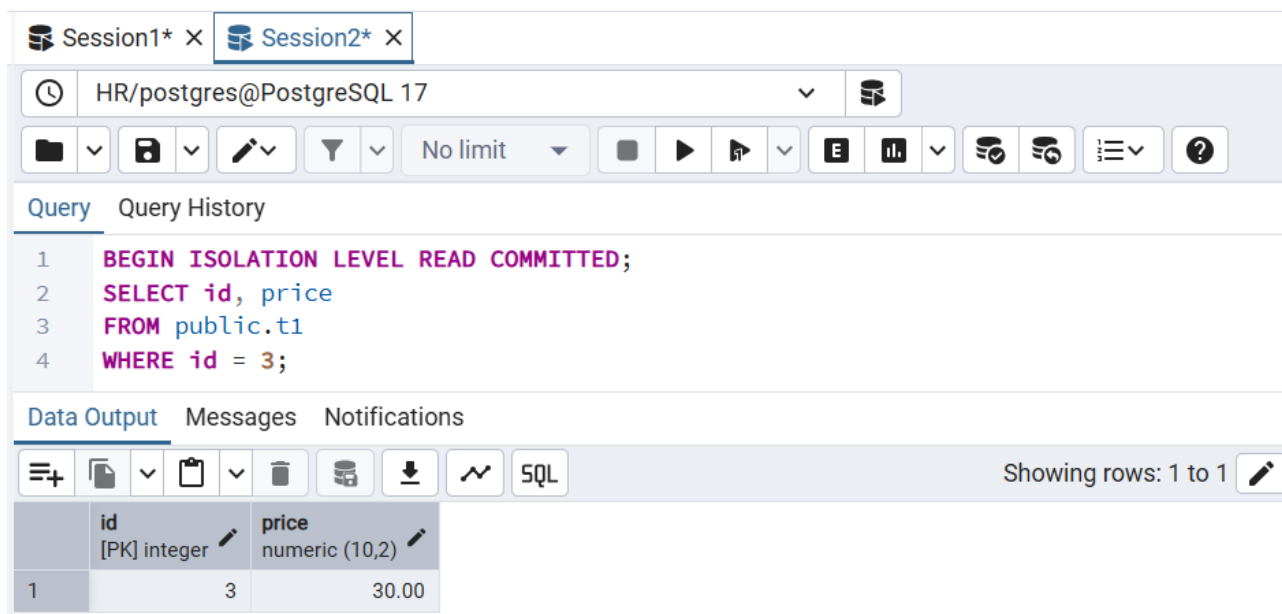


Рисунок 26 — Начало второй транзакции и просмотр данных

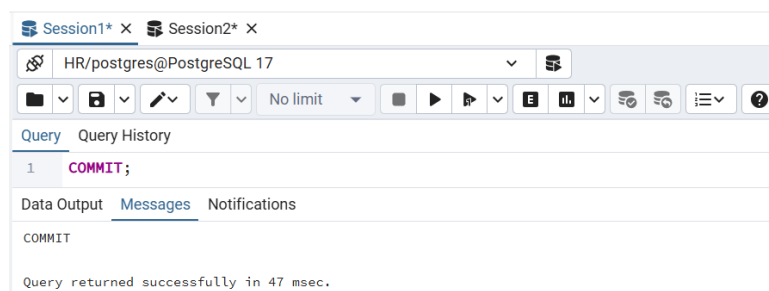


Рисунок 27 — Фиксация первой транзакции

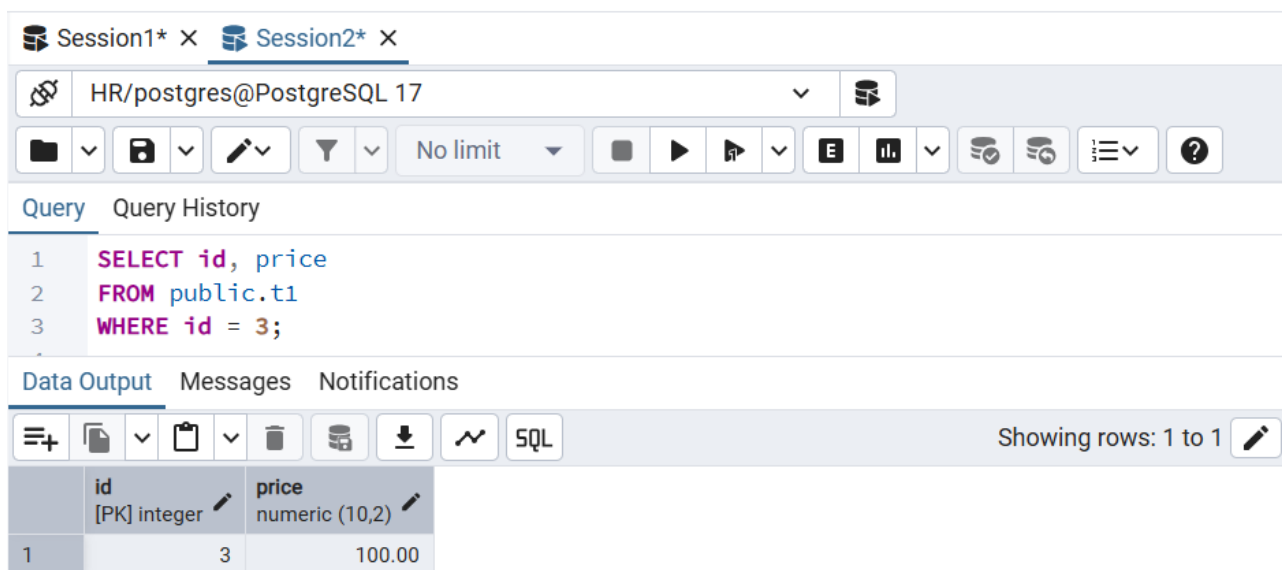


Рисунок 28 — Просмотр данных во второй транзакции

4 ЗАДАНИЕ 4

В этом задании был рассмотрен уровень изоляции REPEATABLE READ.

Результаты выполнения задания для проверки неповторяемого чтения представлены на рисунках 29 - 31.

Как и ожидалось, при повторном выполнении запроса в Session1 возвращается та же строка, что и при первом чтении.

The screenshot shows a PostgreSQL client interface with two sessions: Session1* and Session2*. The active session is Session1*. The connection is to HR/postgres@PostgreSQL 17. The query editor shows the following SQL:

```
1 BEGIN ISOLATION LEVEL REPEATABLE READ;  
2 SELECT id, price  
3 FROM public.t1  
4 WHERE id = 3;
```

The Data Output tab is selected, showing the results of the query:

	id [PK] integer	price numeric (10,2)
1	3	100.00

Showing rows: 1 to 1

Рисунок 29 — Запуск транзакции с уровнем изоляции REPEATABLE READ

The screenshot shows the same PostgreSQL client interface, but now Session2* is the active session. The connection is to HR/postgres@PostgreSQL 17. The query editor shows the following SQL:

```
1 BEGIN;  
2 UPDATE public.t1  
3 SET price = 1  
4 WHERE id = 3;  
5 COMMIT;
```

The Messages tab is selected, showing the following messages:

```
COMMIT  
  
Query returned successfully in 37 msec.
```

Рисунок 30 — Изменение данных другой транзакцией

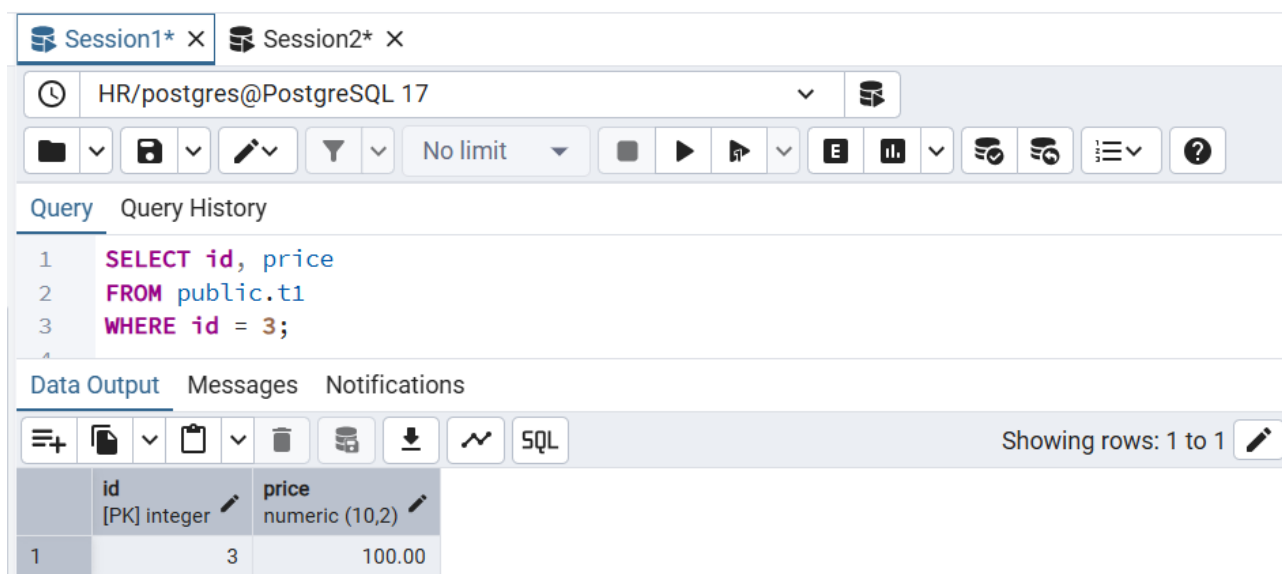


Рисунок 31 — Просмотр данных в первой транзакции

Результаты выполнения задания для проверки фантомного чтения представлены на рисунках 32 - 34.

Как и ожидалось, фантомное чтение не возникает: повторные запросы внутри одной транзакции видят один и тот же набор строк, даже если параллельная транзакция добавляет новые строки, подходящие под условие запроса.

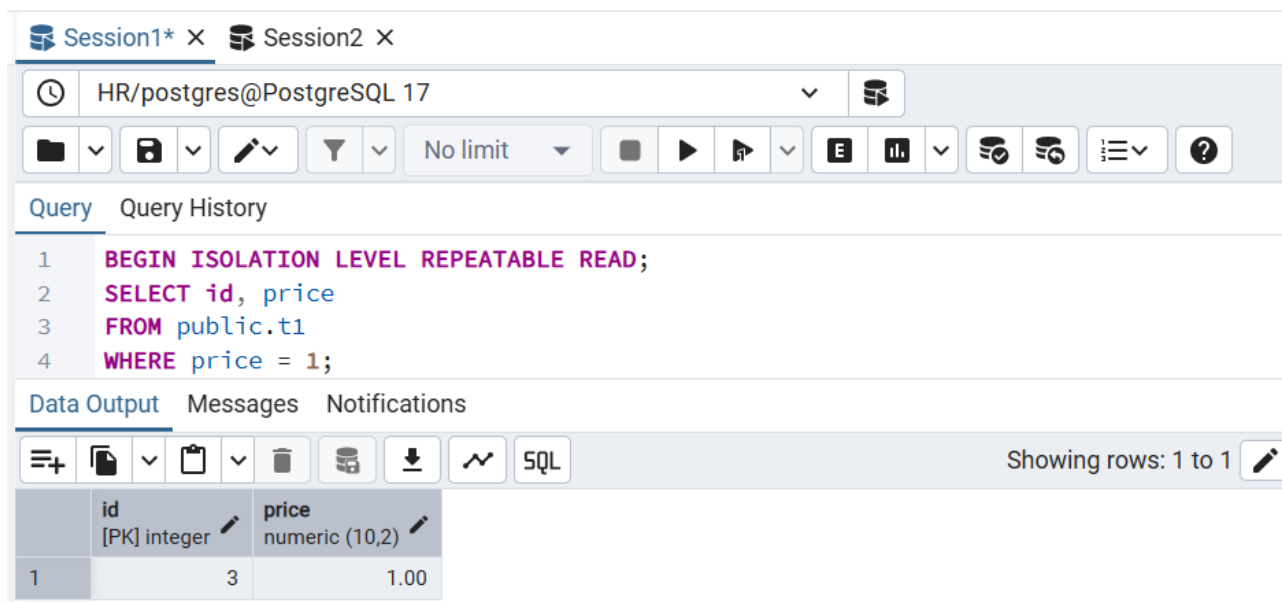


Рисунок 32 — Запуск транзакции с уровнем изоляции REPEATABLE READ

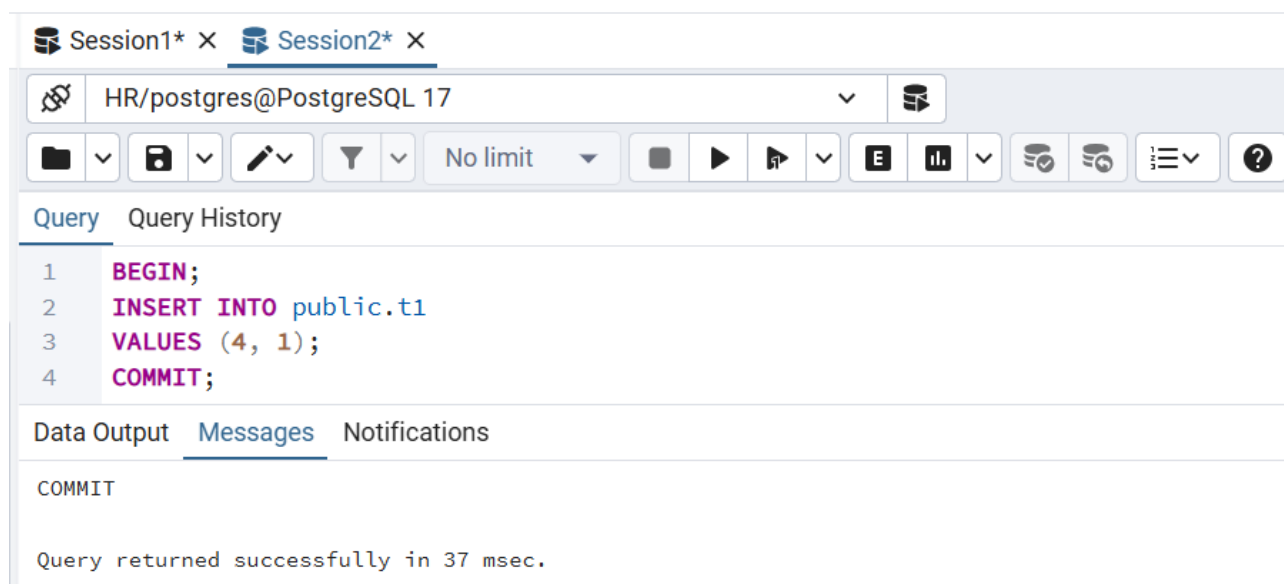


Рисунок 33 — Добавление строки другой транзакцией

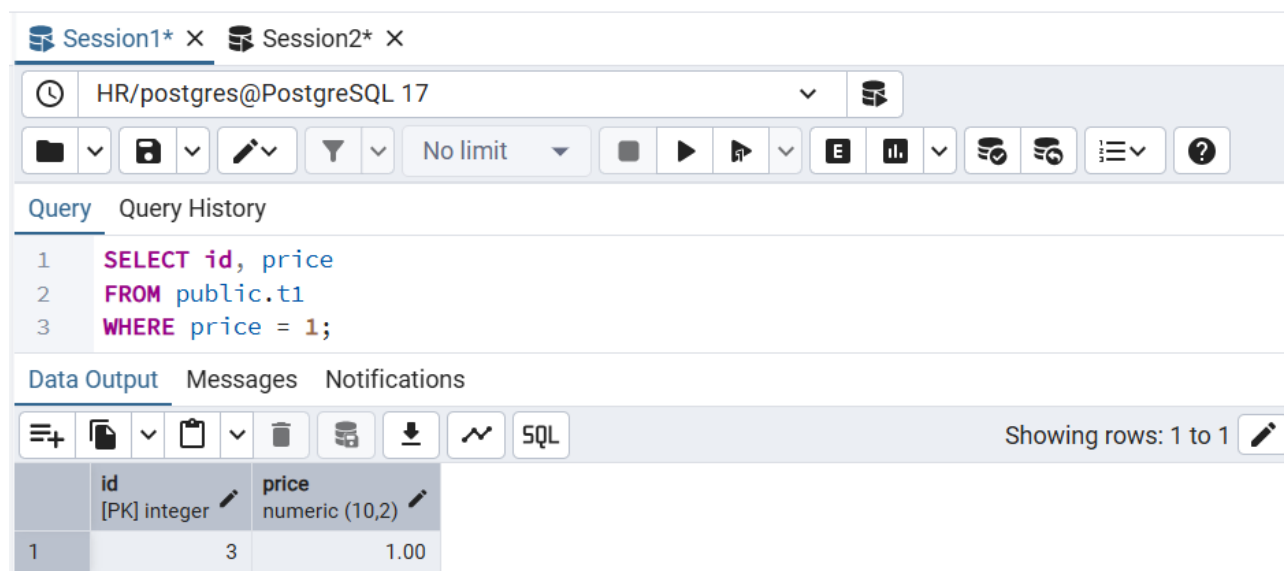


Рисунок 34 — Просмотр данных в первой транзакции

5 ЗАДАНИЕ 5

В этом задании был рассмотрен уровень изоляции `SERIALIZABLE`.

Результаты выполнения задания для проверки представлены на рисунках 35 - 39.

Как и ожидалось, Session2 ожидает закрытия транзакции в Session1 так как присутствует совместная блокировка строки, а после окончания блокировки заканчивается с ошибкой.

The screenshot shows a PostgreSQL client interface with two sessions. Session1* is active and contains a transaction that has started with `BEGIN ISOLATION LEVEL SERIALIZABLE;` and is performing an `UPDATE` on `public.t1` to set `price = 111` where `id = 1`. Session2 is also active and contains a query that selects `id` and `price` from `public.t1` where `id = 1`. The interface includes a toolbar with various icons for file operations, query execution, and data viewing. The 'Data Output' tab is selected, showing a table with columns `id` and `price`. The table has one row with `id = 1` and `price = 111.00`.

```
1 BEGIN ISOLATION LEVEL SERIALIZABLE;
2
3 -- Обновляем строку с id = 1
4 UPDATE public.t1
5 SET price = 111
6 WHERE id = 1;
7
8 -- Читаем строку с id = 1
9 SELECT id, price
10 FROM public.t1
11 WHERE id = 1;
```

id [PK] integer	price numeric (10,2)
1	111.00

Рисунок 35 — Запуск транзакции и обновление данных

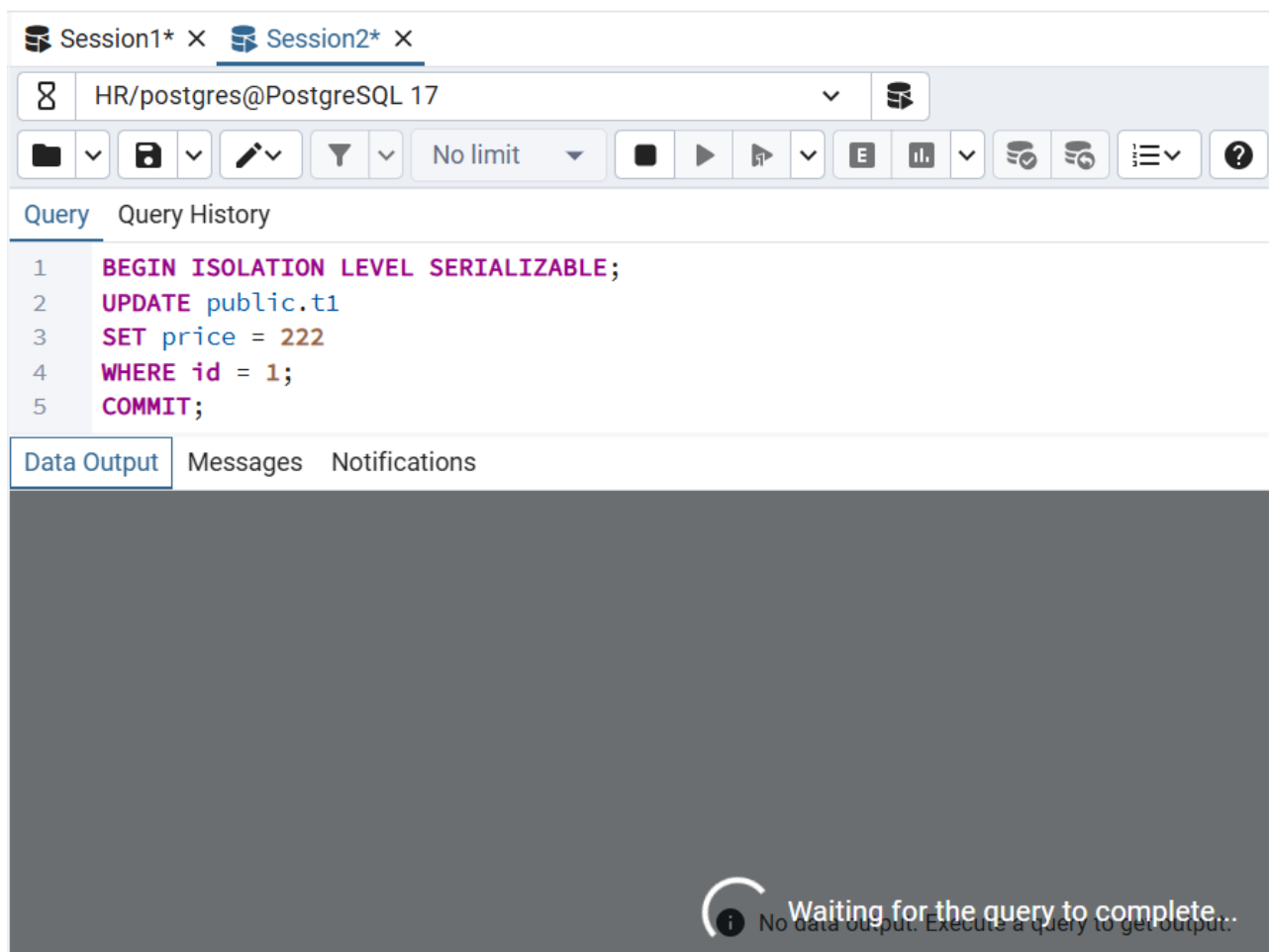


Рисунок 36 — Запуск второй транзакции с обновлением данных

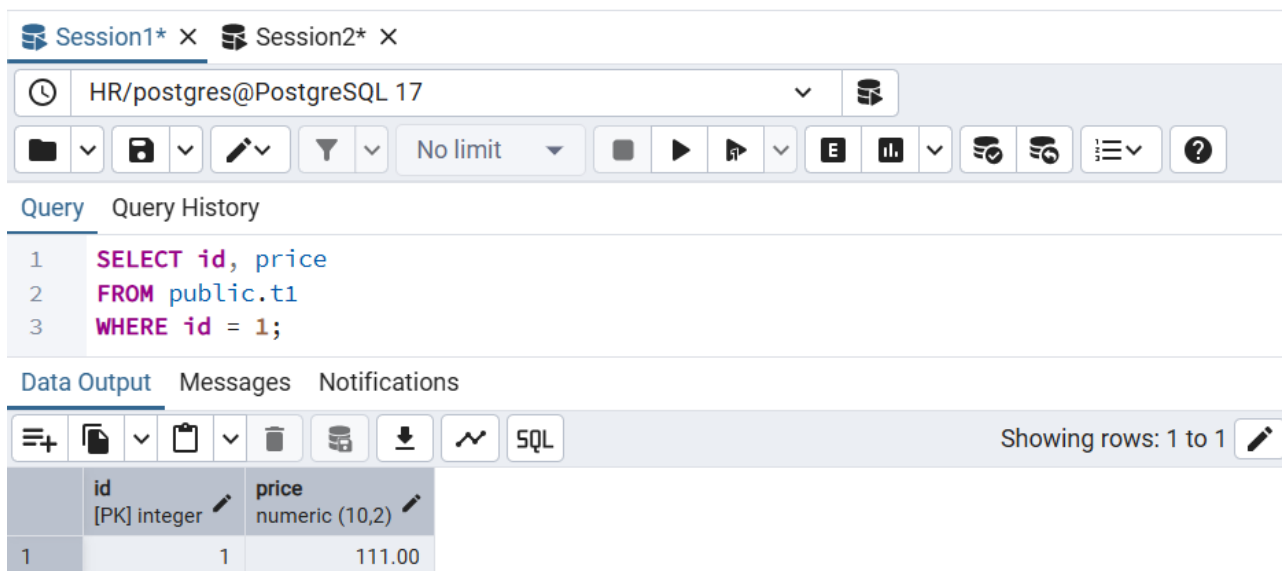


Рисунок 37 — Просмотр данных в первой транзакции

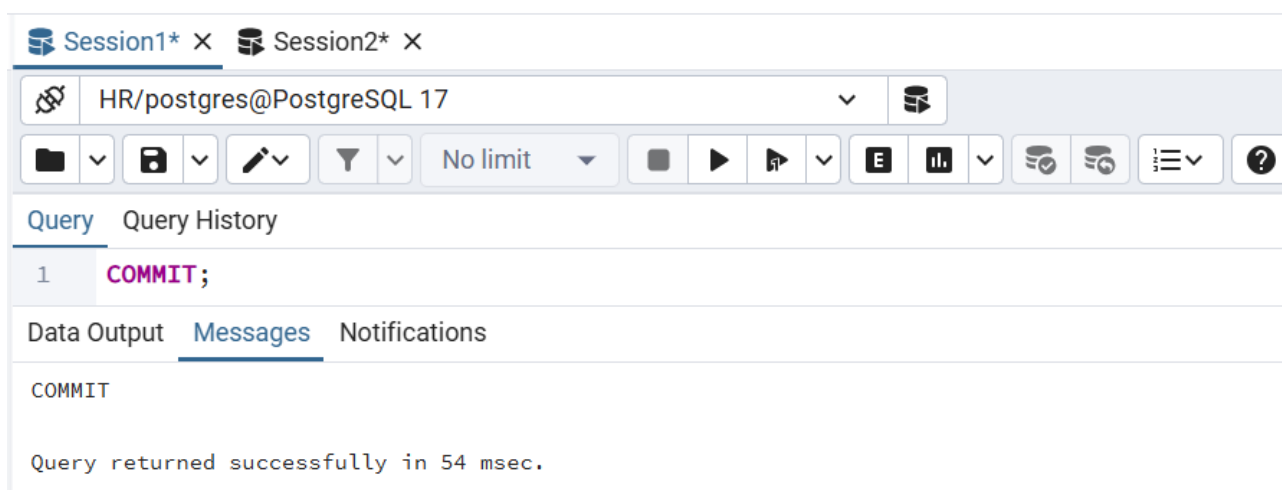


Рисунок 38 — Завершение первой транзакции

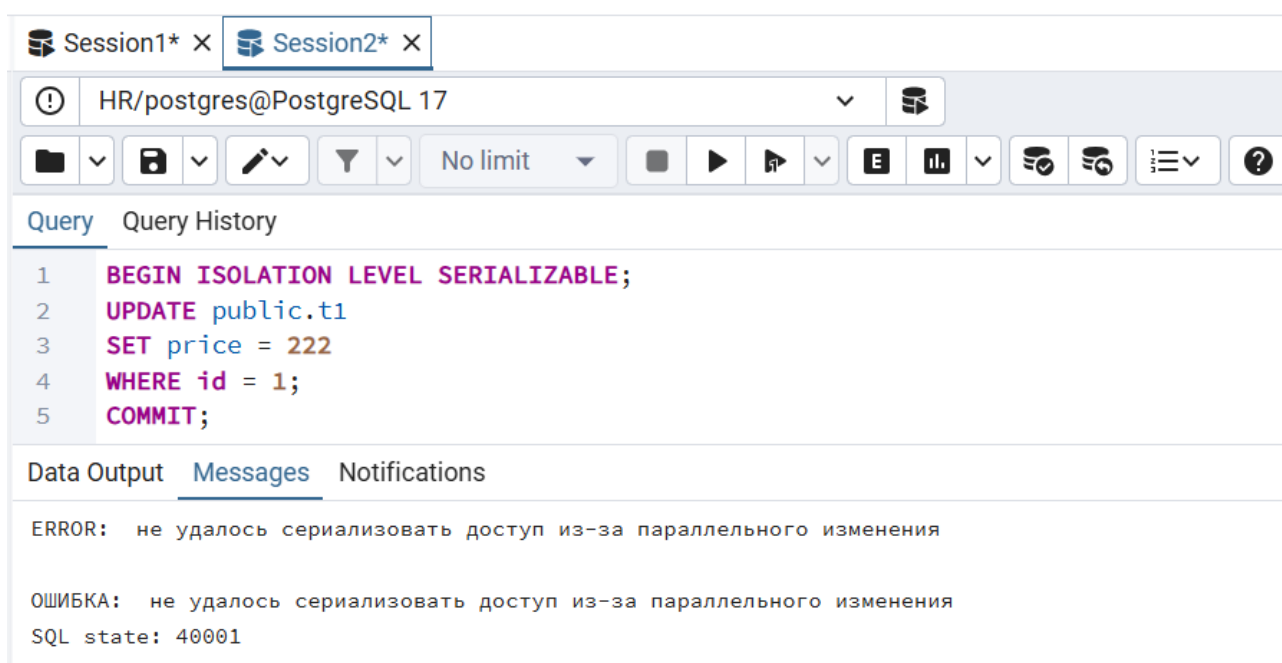


Рисунок 39 — Ошибка после разблокировании второй транзакции

ЗАКЛЮЧЕНИЕ

В ходе работы все задания были выполнены успешно. Для каждого задания написаны и протестированы запросы. Все действия выполнены согласно заданию.

При работе оказалось, что pgAdmin4 выводит только последний результат оператора SELECT, поэтому пришлось разбивать скрипты и запускать частями. Также в настройках был отключён auto commit. Основной и единственной сложностью было выполнить все нужные команды последовательно и не ошибиться: при неправильном вводе часто приходилось начинать задание сначала, чтобы воссоздать необходимую ситуацию.

За время работы я изучил основы транзакций, фиксаций, откатов, точек сохранения. Рассмотрел все три уровня изоляции в PostgreSQL. Для закрепления придумал, где они могут использоваться.